AI ASSITED CODING LAB – 1

Lab Experiment: Prompt Engineering – Improving Prompts and Context Management

Name : G.Sreeja

Roll No:2503a52692

Batch No : 24

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Prompt : Generate a python function that checks whether a given number is a palindrome or not. If true then it is a palindrome and if no then it is not a palindrome

#code

```python
def check_palindrome(number):
    """Print whether the given number is a palindrome."""
    text = str(number)
    if text == text[::-1]:
        print("It is a palindrome")
    else:
        print("It is not a palindrome")


number = int(input("Enter a number: "))
check_palindrome(number)

```

#output

```
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: -7007
It is not a palindrome
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Use
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: 7007
It is a palindrome
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Use
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: 2007
It is not a palindrome
```

Logical Errors & Missing Edge Cases (Palindrome Program):

1.Negative numbers fail due to the - sign, not handled explicitly

2.No input validation (crashes on non-numeric input)

3.Function only prints, doesn't return True/False

4.Edge cases like 0 or trailing zeros not tested

Question 2: One-Shot Prompting (Factorial Calculation)

Write a **one-shot prompt** by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Prompt : Write a Python function called factorial that computes the factorial of a given number.Example:Input: 5 → Output: 120

Generate the function based on this example.

#code

```python
def factorial(number):
    """Return the factorial of a non-negative integer."""
    if number < 0:
        raise ValueError("factorial is not defined for negative numbers")

    result = 1
    for value in range(2, number + 1):
        result *= value
    return result


user_input = int(input("Enter a number: "))
print(factorial(user_input))
```

#output

Comparison with zero-shot solution:

Zero-shot version usually just does the loop without checking negative numbers — it may crash unexpectedly or give wrong output for n < 0.

One-shot version (yours) explicitly raises a ValueError for negative numbers, making the error controlled and clear instead of a random crash.

Improvements in clarity & correctness:

```
Enter a number: 9
362880
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIAC/exp2.py
Enter a number: -4
Traceback (most recent call last):
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 13, in <module>
    print(factorial(user_input))
          ~~~~~~~~~^^^^^^^^^^^^
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 4, in factorial
    raise ValueError("factorial is not defined for negative numbers")
ValueError: factorial is not defined for negative numbers
PS C:\Users\pc\OneDrive\AIAC> 4
4
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIAC/exp2.py
Enter a number: 11
39916800
```

1.Handles negative input properly with a meaningful error message

2.Iterative loop (not recursive) — safer for large numbers

3.Docstring added, explaining what the function does

4.Output matches the given example exactly (5 → 120), confirming correctness

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a **few-shot prompt** by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number

- Input: 123 → Output: Not an Armstrong Number

prompt: Write a Python function called check_armstrong that checks whether a given number is an Armstrong number.

Examples:Input: 153 → Output: ArmstrongNumber,

Input: 370 → Output: Armstrong Number,

Input: 123 → Output: Not an Armstrong Number

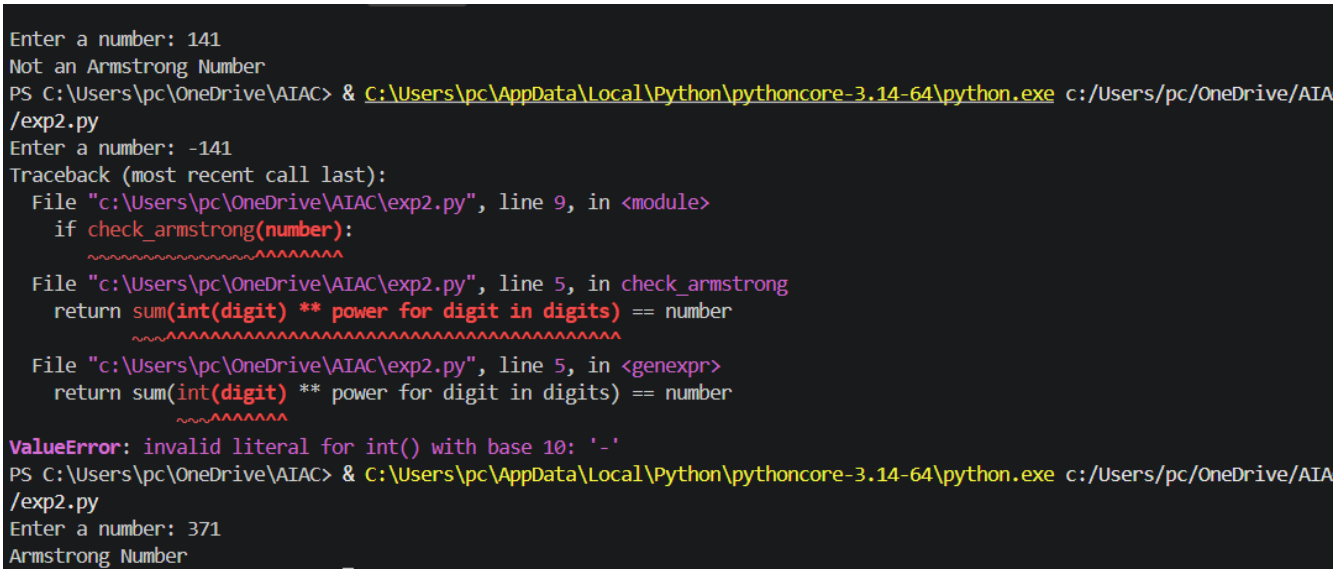
Generate the function based on these examples**.**

#code

```python
def check_armstrong(number):
    """Return whether number is an Armstrong number."""
    digits = str(number)
    power = len(digits)
    return sum(int(digit) ** power for digit in digits) == number


number = int(input("Enter a number: "))
if check_armstrong(number):
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")
```

#output



Logical error found: The code doesn't handle negative numbers. str(-141) includes the - sign, and int('-') fails when iterating character-by-character — causing an unhandled crash instead of a clean "Not an Armstrong Number" response.

(Optional Extension)

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a **context-managed prompt** with clear instructions and constraints to generate an optimized Python program that classifies a number as **prime, composite, or neither**.

Prompt: Write a Python program that asks the user to enter a number and tells whether it is prime, composite, or neither. Make sure the code checks for valid input and uses an efficient method (don't check every number up to n, just check up to its square root). Also skip checking even numbers to make it faster.

#code

exp2.py > ...

```python
import math


def classify_number(number):
    """Return whether number is prime, composite, or neither."""
    if number <= 1:
        return "neither"
    if number == 2:
        return "prime"
    if number % 2 == 0:
        return "composite"

    limit = math.isqrt(number)
    divisor = 3
    while divisor <= limit:
        if number % divisor == 0:
            return "composite"
        divisor += 2
    return "prime"


try:
    number = int(input("Enter an integer: ").strip())
except ValueError:
    print("Invalid input. Please enter a whole number.")
else:
    print(f"{number} is {classify_number(number)}.")
```

#output

```
/exp2.py
Enter an integer: 8
8 is composite.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: -5
-5 is neither.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: 5
5 is prime.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: 1
1 is neither.
```

Task analysis:

1.Optimized for efficiency — uses math.isqrt(number) for O(√n) time, checks only odd divisors after eliminating even numbers early.

2.Comparison with earlier strategies — zero-shot versions typically lacked validation and used slower O(n) loops; this context-managed version is faster, safer, and handles edge cases (negative numbers, 0, 1) correctly out of the box.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Prompt: Write a python function for perfect number that checks whether agiven number is a perfect number or not if yes then it is a perfect number if no then it is not a perfect number

#code

```python
def is_perfect_number(number):
    """Return True if number equals the sum of its proper divisors."""
    if number <= 1:
        return False

    divisor_sum = 1
    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            divisor_sum += divisor
            paired_divisor = number // divisor
            if paired_divisor != divisor:
                divisor_sum += paired_divisor

    return divisor_sum == number


number = int(input("Enter a number: "))
if is_perfect_number(number):
    print("It is a perfect number.")
else:
    print("It is not a perfect number.")
```

#output

```
/exp2.py
Enter a number: 6
It is a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDriv
/exp2.py
Enter a number: -6
It is not a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDriv
/exp2.py
Enter a number: 6.0
Traceback (most recent call last):
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 17, in <module>
    number = int(input("Enter a number: "))
ValueError: invalid literal for int() with base 10: '6.0'
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDriv
/exp2.py
Enter a number: 100
It is not a perfect number.
```

Missing Conditions / Inefficiencies:

Crashes on decimal input (6.0) — int() can't parse a string with a decimal point

No try/except for invalid/non-numeric input

Already uses √n optimization (efficient), but no upper limit check for very large numbers

No explicit test/comment for edge case number == 1

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

1.Input: 8 → Output: Even

2.Input: 15 → Output: Odd

3.Input: 0 → Output: Even

#prompt: Write a Python function called check_even_odd that determines whether a given number is even or odd, including proper input validation. Examples: Input: 8 → Output: Even Input: 15 → Output: Odd Input: 0 → Output: Even Generate the function based on these examples.

#code

exp2.py > check_even_odd

```python
def check_even_odd(number):
    """Return whether an integer is even or odd."""
    if isinstance(number, bool) or not isinstance(number, int):
        raise ValueError("Please enter a valid integer.")

    return "Even" if number % 2 == 0 else "Odd"


try:
    user_number = int(input("Enter an integer: "))
    print(check_even_odd(user_number))
except ValueError as error:
    print(error)
```

#output

```
/exp2.py
Enter a number: 100
It is not a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 89
Odd
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 78
Even
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: -10
Even
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 23.8
invalid literal for int() with base 10: '23.8'
```

Short analysis:

Examples improved output clarity — AI matched exact string format ("Even"/"Odd") instead of returning booleans.

Including 0 as an example pushed the AI to handle the zero edge case correctly.

Negative numbers work correctly since Python's % handles negatives properly for even/odd checks.

Input validation was added proactively due to the task instruction, even without an explicit invalid-input example.

Conclusion:

This lab assignment provided hands-on experience with different prompt engineering strategies — zero-shot, one-shot, few-shot, and context-managed prompting — applied to common programming problems like palindrome checking, factorial calculation, Armstrong number verification, prime/composite classification, and perfect number checking. Through testing and comparing AI-generated code across these strategies, it became clear that prompt specificity directly impacts code quality: zero-shot prompts often produced functional but edge-case-fragile code, while few-shot and context-managed prompts — by providing examples and explicit constraints — consistently generated more robust, efficient, and well-validated programs. This exercise highlighted that clear, well-structured prompts are essential for extracting reliable and production-ready code from AI models.